Practical Session 5: Arrays and ArrayLists

Student-Centered, Self-Paced Lab

➢ Learning Outcomes Covered

By the end of this practical session, you will be able to:

1. **Analyze** array algorithms to determine their time complexity (linear vs quadratic).
2. **Evaluate** the trade-offs between using arrays and ArrayLists for different scenarios (memory, performance, flexibility).
3. **Debug** array index out-of-bounds errors by simulating loop conditions.
4. **Design** an application that uses both arrays and ArrayLists to manage a collection (e.g., student roster).

Time Required: 120–150 minutes
Instructor Intervention: None – you will figure it out yourself!

➢ Before You Begin

Ensure you have completed **Week 1, 2, 3, and 4 practical sessions**. You will need:

- VS Code open
- Your java-course folder with week4 subfolder
- Git configured and connected to GitHub

➢ Your Roadmap

- **Task 1:** Introduction to Arrays
- **Task 2:** Array Algorithms – Search, Max, Min, Average
- **Task 3:** Multidimensional Arrays
- **Task 4:** Introduction to ArrayLists
- **Task 5:** ArrayList vs Array – Trade-off Analysis

- 🟦 **Task 6:** Debugging Array Index Errors
- 🟫 **Task 7:** Integrated Project – Student Roster
- ⬛ **Task 8:** Push to GitHub

TASK 1: Introduction to Arrays

Objective

Understand how arrays work, including declaration, initialization, access, and iteration.

Step 1.1: Create a New Folder

1. Open **VS Code**.
2. Open your java-course folder.
3. Right-click the java-course folder → **New Folder**.
4. Name it: week5
5. Press Enter.

Step 1.2: Create the File

1. Right-click the week5 folder → **New File**.
2. Name the file: ArrayBasics.java
3. Press Enter.

Step 1.3: Write the Code

Copy and paste this code into ArrayBasics.java:

java

```
// This program demonstrates the basics of arrays in Java.


public class ArrayBasics {
   public static void main(String[ ] args) {


    // ==========================================================
```

```
// SECTION 1: DECLARING AND INITIALIZING ARRAYS
// ============================================================

// Method 1: Declare array with size, then assign values individually.
// int[] means "an array of integers".
// new int[5] creates space for 5 integers (all initialized to 0).
int[] numbers = new int[5];

// Assign values to each element using the index.
// Array indices start at 0.
numbers[0] = 10;  // First element (index 0)
numbers[1] = 20;  // Second element (index 1)
numbers[2] = 30;  // Third element (index 2)
numbers[3] = 40;  // Fourth element (index 3)
numbers[4] = 50;  // Fifth element (index 4)

// Method 2: Declare and initialize in one line using curly braces.
// The array size is determined automatically by the number of values.
int[] scores = {85, 92, 78, 95, 88, 76, 90};

// Method 3: Declare a String array.
String [ ] names = {"Alice", "Bob", "Charlie", "Diana"};
// ============================================================
// SECTION 2: ACCESSING ARRAY ELEMENTS
// ============================================================
```

```
System.out.println("=== ACCESSING ARRAY ELEMENTS ===");
// Access individual elements using index.
System.out.println("First number: " + numbers[0]);   // 10
System.out.println("Third number: " + numbers[2]);   // 30
System.out.println("Last number: " + numbers[4]);    // 50

// Access the length of the array using the length property.
System.out.println("Length of numbers array: " + numbers.length);
System.out.println("Length of scores array: " + scores.length);
System.out.println("Length of names array: " + names.length);

// ============================================================
// SECTION 3: ITERATING WITH A TRADITIONAL FOR LOOP
// ============================================================

System.out.println("\n=== TRADITIONAL FOR LOOP ===");

// Loop through the array using the index.
// i starts at 0 and goes up to length - 1.
for (int i = 0; i < numbers.length; i++) {
    // Access element at index i.
    System.out.println("numbers[" + i + "] = " + numbers[i]);
}

// ============================================================
// SECTION 4: ITERATING WITH AN ENHANCED FOR LOOP
```

```
// ============================================================

System.out.println("\n=== ENHANCED FOR LOOP ===");

// Enhanced for loop (for-each) iterates through all elements.
// No need for index; variable gets each element in turn.
for (int num : numbers) {
    System.out.println("Value: " + num);
}

System.out.println("\n=== NAMES ARRAY ===");

// Enhanced for loop with strings.
for (String name : names) {
    System.out.println("Name: " + name);
}

// ============================================================
// SECTION 5: DEFAULT VALUES
// ============================================================

System.out.println("\n=== DEFAULT VALUES ===");

// Create an array without assigning values.
int[] defaultInts = new int[3];
```

```
        // Default value for int array is 0.
        System.out.println("Default int: " + defaultInts[0]); // 0

        // Create a boolean array.
        boolean[] defaultBooleans = new boolean[3];

        // Default value for boolean array is false.
        System.out.println("Default boolean: " + defaultBooleans[0]); // false

        // Create a String array.
        String[ ] defaultStrings = new String[3];

        // Default value for String array is null.
        System.out.println("Default String: " + defaultStrings[0]); // null
    }
}
```

Step 1.4: Run the Program

1. Press Ctrl + S to save.
2. Click the **Run** button (▶️).
3. Observe the output.

Expected Output:

```
=== ACCESSING ARRAY ELEMENTS ===
First number: 10
Third number: 30
Last number: 50
Length of numbers array: 5
```

Length of scores array: 7

Length of names array: 4

=== TRADITIONAL FOR LOOP ===

numbers[0] = 10

numbers[1] = 20

numbers[2] = 30

numbers[3] = 40

numbers[4] = 50

=== ENHANCED FOR LOOP ===

Value: 10

Value: 20

Value: 30

Value: 40

Value: 50

=== NAMES ARRAY ===

Name: Alice

Name: Bob

Name: Charlie

Name: Diana

=== DEFAULT VALUES ===

Default int: 0

Default boolean: false

Default String: null

Step 1.5: Answer These Questions

1. What index does the first element of an array have?
2. What is the last index of an array of length 5?
3. How do you get the length of an array?
4. What is the default value for an int array element?
5. What is the difference between the traditional for loop and the enhanced for loop?

✅ **Task 1 Complete**

- □ I created ArrayBasics.java
- □ I ran the program successfully
- □ I answered the questions

TASK 2: Array Algorithms – Search, Max, Min, Average

Objective

Write algorithms that process arrays and analyze their time complexity.

Step 2.1: Create the File

Create a new file called ArrayAlgorithms.java in the week5 folder.

Step 2.2: Write the Code

java

```
// This program demonstrates common array algorithms:
// - Finding the maximum value
// - Finding the minimum value
// - Calculating the average
// - Linear search


public class ArrayAlgorithms {
```

```
public static void main(String[] args) {

    // Declare and initialize an array of marks.
    int[] marks = {65, 72, 48, 91, 57, 83, 69, 75, 52, 88};

    // Print the array for reference.
    System.out.print("Marks: ");
    for (int mark : marks) {
        System.out.print(mark + " ");
    }
    System.out.println("\n");

    // ============================================================
    // ALGORITHM 1: FIND THE MAXIMUM VALUE (O(n))
    // ============================================================

    System.out.println("=== FIND MAXIMUM ===");

    // Assume the first element is the maximum.
    int max = marks[0];

    // Iteration counter to analyze efficiency.
    int maxIterations = 0;

    // Loop through all elements.
    for (int i = 0; i < marks.length; i++) {
```

```
    // Increment the iteration counter.
    maxIterations++;

    // If the current element is greater than max, update max.
    if (marks[i] > max) {
        max = marks[i];
    }
}

System.out.println("Maximum mark: " + max);
System.out.println("Iterations: " + maxIterations);
System.out.println("Time Complexity: O(n)");

// ============================================================
// ALGORITHM 2: FIND THE MINIMUM VALUE (O(n))
// ============================================================

System.out.println("\n=== FIND MINIMUM ===");

// Assume the first element is the minimum.
int min = marks[0];

// Iteration counter.
int minIterations = 0;

// Loop through all elements.
```

```
for (int i = 0; i < marks.length; i++) {
    minIterations++;
    // If the current element is less than min, update min.
    if (marks[i] < min) {
        min = marks[i];
    }
}

System.out.println("Minimum mark: " + min);
System.out.println("Iterations: " + minIterations);
System.out.println("Time Complexity: O(n)");

// ============================================================
// ALGORITHM 3: CALCULATE THE AVERAGE (O(n))
// ============================================================

System.out.println("\n=== CALCULATE AVERAGE ===");

// Variable to hold the sum of all marks.
int sum = 0;

// Loop through the array and add each element to sum.
for (int i = 0; i < marks.length; i++) {
    sum += marks[i];
}
```

```
// Calculate the average as a double.
double average = (double) sum / marks.length;

System.out.println("Sum: " + sum);
System.out.println("Number of marks: " + marks.length);
System.out.printf("Average: %.2f%n", average);
System.out.println("Time Complexity: O(n)");

// ============================================================
// ALGORITHM 4: LINEAR SEARCH (O(n))
// ============================================================

System.out.println("\n=== LINEAR SEARCH ===");

// The value we are looking for.
int target = 83;

// Variable to store the position (index). -1 means not found.
int foundIndex = -1;

// Iteration counter.
int searchIterations = 0;

// Loop through the array looking for the target.
for (int i = 0; i < marks.length; i++) {
    searchIterations++;
```

```
    if (marks[i] == target) {
        foundIndex = i;
        break; // Exit loop once found (early exit optimization)
    }
}

// Report the result.
if (foundIndex != -1) {
    System.out.println("Found " + target + " at index " + foundIndex);
} else {
    System.out.println(target + " not found in the array.");
}

System.out.println("Iterations: " + searchIterations);
System.out.println("Time Complexity: O(n) - best case O(1), worst case O(n)");

// ============================================================
// ALGORITHM 5: COUNT ELEMENTS ABOVE THRESHOLD (O(n))
// ============================================================

System.out.println("\n=== COUNT ABOVE THRESHOLD ===");

// Threshold value (pass mark).
int threshold = 70;

// Counter for elements above threshold.
```

```
int count = 0;

// Loop through the array.
for (int mark : marks) {
   // Increment count if the mark is >= threshold.
   if (mark >= threshold) {
      count++;
   }
}

System.out.println("Marks above or equal to " + threshold + ": " + count);
System.out.println("Total marks: " + marks.length);
System.out.println("Time Complexity: O(n)");

// ============================================================
// COMPLEXITY SUMMARY
// ============================================================

System.out.println("\n=== COMPLEXITY SUMMARY ===");
System.out.println("Algorithm          | Iterations | Complexity");
System.out.println("-------------------|------------|-----------");
System.out.println("Find Maximum       | " + maxIterations + "         | O(n)");
System.out.println("Find Minimum       | " + minIterations + "         | O(n)");
System.out.println("Average            | " + marks.length + "         | O(n)");
System.out.println("Linear Search      | " + searchIterations + "         | O(n)");
System.out.println("Count Above        | " + marks.length + "         | O(n)");
```

```
    }
}
```

Step 2.3: Run the Program

Save and run. Observe the output.

Expected Output:

```
Marks: 65 72 48 91 57 83 69 75 52 88

=== FIND MAXIMUM ===
Maximum mark: 91
Iterations: 10
Time Complexity: O(n)

=== FIND MINIMUM ===
Minimum mark: 48
Iterations: 10
Time Complexity: O(n)

=== CALCULATE AVERAGE ===
Sum: 700
Number of marks: 10
Average: 70.00
Time Complexity: O(n)

=== LINEAR SEARCH ===
Found 83 at index 5
Iterations: 6
```

Time Complexity: O(n) - best case O(1), worst case O(n)

=== COUNT ABOVE THRESHOLD ===

Marks above or equal to 70: 5

Total marks: 10

Time Complexity: O(n)

=== COMPLEXITY SUMMARY ===

Algorithm | Iterations | Complexity

---|---|---

Find Maximum | 10 | O(n)

Find Minimum | 10 | O(n)

Average | 10 | O(n)

Linear Search | 6 | O(n)

Count Above | 10 | O(n)

Step 2.4: Answer These Questions

1. Why did the linear search take only 6 iterations instead of 10?
2. Why are all these algorithms O(n)?
3. What would happen if the target was at the last index? How many iterations?
4. What would happen if the target was not in the array? How many iterations?

✅ **Task 2 Complete**

- □ I created ArrayAlgorithms.java
- □ I ran the program successfully
- □ I answered the questions

TASK 3: Multidimensional Arrays

Objective

Understand and use 2D arrays to store tabular data.

Step 3.1: Create the File

Create a new file called MatrixDemo.java in the week5 folder.

Step 3.2: Write the Code

java

```
// This program demonstrates two-dimensional (2D) arrays in Java.
// 2D arrays are useful for tables, grids, and matrices.


public class MatrixDemo {
    public static void main(String[ ] args) {


        // ============================================================
        // SECTION 1: DECLARING AND INITIALIZING A 2D ARRAY
        // ============================================================


        // Method 1: Using curly braces to initialize a 2D array.
        // Each inner curly brace represents a row.
        int[][] matrix = {
            {1, 2, 3},  // Row 0
            {4, 5, 6},  // Row 1
            {7, 8, 9}   // Row 2
        };
```

```
// Method 2: Declaring with size and assigning later.
// int[][] table = new int[3][4]; // 3 rows, 4 columns

// ============================================================
// SECTION 2: ACCESSING ELEMENTS
// ============================================================

System.out.println("=== ACCESSING ELEMENTS ===");

// Access element at row 0, column 0.
System.out.println("matrix[0][0] = " + matrix[0][0]); // 1

// Access element at row 1, column 2.
System.out.println("matrix[1][2] = " + matrix[1][2]); // 6

// Access element at row 2, column 1.
System.out.println("matrix[2][1] = " + matrix[2][1]); // 8

// Number of rows = matrix.length
System.out.println("Number of rows: " + matrix.length);

// Number of columns in row 0 = matrix[0].length
System.out.println("Number of columns in row 0: " + matrix[0].length);

// ============================================================
// SECTION 3: PRINTING THE MATRIX WITH NESTED LOOPS
```

```
// ============================================================

System.out.println("\n=== PRINTING THE MATRIX ===");

// Outer loop iterates through rows.
for (int i = 0; i < matrix.length; i++) {

   // Inner loop iterates through columns of row i.
   for (int j = 0; j < matrix[i].length; j++) {
      // Print each element with spacing.
      System.out.print(matrix[i][j] + " ");
   }

   // Move to the next line after printing a row.
   System.out.println();
}

// ============================================================
// SECTION 4: ENHANCED FOR LOOP WITH 2D ARRAYS
// ============================================================

System.out.println("\n=== ENHANCED FOR LOOP ===");

// Outer for-each iterates through each row (which is an int[]).
for (int[ ] row : matrix) {
   // Inner for-each iterates through each element in the row.
```

```
    for (int value : row) {
        System.out.print(value + "\t");
    }
    System.out.println();
}

// ============================================================
// SECTION 5: SUMMING ELEMENTS IN A 2D ARRAY
// ============================================================

System.out.println("\n=== SUM OF ALL ELEMENTS ===");

// Variable to hold the total sum.
int totalSum = 0;

// Outer loop through rows.
for (int i = 0; i < matrix.length; i++) {
    // Inner loop through columns.
    for (int j = 0; j < matrix[i].length; j++) {
        // Add each element to the sum.
        totalSum += matrix[i][j];
    }
}

System.out.println("Sum of all elements: " + totalSum);
```

```
// ============================================================
// SECTION 6: ROW SUMS AND COLUMN SUMS
// ============================================================

System.out.println("\n=== ROW SUMS ===");

// Loop through each row.
for (int i = 0; i < matrix.length; i++) {
    // Variable to hold the sum of this row.
    int rowSum = 0;

    // Loop through each column in this row.
    for (int j = 0; j < matrix[i].length; j++) {
        rowSum += matrix[i][j];
    }

    System.out.println("Sum of row " + i + ": " + rowSum);
}

System.out.println("\n=== COLUMN SUMS ===");

// Assume all rows have the same length.
int columns = matrix[0].length;

// Loop through each column.
for (int j = 0; j < columns; j++) {
```

```
    // Variable to hold the sum of this column.
    int colSum = 0;

    // Loop through each row in this column.
    for (int i = 0; i < matrix.length; i++) {
        colSum += matrix[i][j];
    }

    System.out.println("Sum of column " + j + ": " + colSum);
}

// ============================================================
// SECTION 7: REAL-WORLD EXAMPLE – STUDENT MARKS TABLE
// ============================================================

System.out.println("\n=== STUDENT MARKS TABLE ===");

// 2D array with student marks: rows = students, columns = subjects.
int[ ][ ] studentMarks = {
    {85, 90, 78},  // Alice: Math, English, Science
    {72, 88, 91},  // Bob
    {65, 75, 80}   // Charlie
};

String[ ] studentNames = {"Alice", "Bob", "Charlie"};
String[ ] subjects = {"Math", "English", "Science"};
```

```
// Print table header.
System.out.print("Student   ");
for (String subject : subjects) {
   System.out.print(subject + "   ");
}
System.out.println("Average");
System.out.println("----------------------------------------");

// Loop through each student.
for (int i = 0; i < studentMarks.length; i++) {
   // Print student name.
   System.out.print(studentNames[i] + "    ");

   // Variable for total marks.
   int total = 0;

   // Print each mark and accumulate total.
   for (int j = 0; j < studentMarks[i].length; j++) {
      System.out.print(studentMarks[i][j] + "     ");
      total += studentMarks[i][j];
   }

   // Calculate and print the average.
   double avg = (double) total / studentMarks[i].length;
   System.out.printf("%.2f%n", avg);
```

```
    }
  }
}
```

Step 3.3: Run the Program

Save and run. Observe the output.

Expected Output:

text

```
=== ACCESSING ELEMENTS ===
matrix[0][0] = 1
matrix[1][2] = 6
matrix[2][1] = 8
Number of rows: 3
Number of columns in row 0: 3

=== PRINTING THE MATRIX ===
1 2 3
4 5 6
7 8 9

=== ENHANCED FOR LOOP ===
1	2	3
4	5	6
7	8	9

=== SUM OF ALL ELEMENTS ===
Sum of all elements: 45
```

=== ROW SUMS ===

Sum of row 0: 6

Sum of row 1: 15

Sum of row 2: 24

=== COLUMN SUMS ===

Sum of column 0: 12

Sum of column 1: 15

Sum of column 2: 18

=== STUDENT MARKS TABLE ===

Student Math English Science Average

Alice 85 90 78 84.33

Bob 72 88 91 83.67

Charlie 65 75 80 73.33

Step 3.4: Answer These Questions

1. How do you access an element at row 2, column 1?
2. What is the length property of a 2D array?
3. How do you get the number of columns in row 0?
4. What are two real-world use cases for 2D arrays?

✅ **Task 3 Complete**

- □ I created MatrixDemo.java
- □ I ran the program successfully
- □ I answered the questions

TASK 4: Introduction to ArrayLists

Objective

Understand how ArrayLists differ from arrays and use common ArrayList methods.

Step 4.1: Create the File

Create a new file called ArrayListBasics.java in the week5 folder.

Step 4.2: Write the Code

java

```
// This program demonstrates the basics of ArrayLists in Java.
// ArrayList is a dynamic array that can grow and shrink.

// Import the ArrayList class from java.util package.
import java.util.ArrayList;

public class ArrayListBasics {
    public static void main(String[] args) {

        // ============================================================
        // SECTION 1: CREATING AN ARRAYLIST
        // ============================================================

        // Create an ArrayList to store Strings.
        // The <String> is called a generic type parameter.
        // It ensures only Strings can be added to this list.
        ArrayList<String> names = new ArrayList<>();
```

```
// ============================================================
// SECTION 2: ADDING ELEMENTS – add()
// ============================================================

System.out.println("=== ADDING ELEMENTS ===");

// Add elements to the end of the list.
names.add("Alice");
names.add("Bob");
names.add("Charlie");

System.out.println("After adding: " + names);

// Add an element at a specific index.
// This shifts subsequent elements to the right.
names.add(1, "Diana");

System.out.println("After adding Diana at index 1: " + names);

// ============================================================
// SECTION 3: ACCESSING ELEMENTS – get()
// ============================================================

System.out.println("\n=== ACCESSING ELEMENTS ===");

// Get the element at index 0.
```

```
System.out.println("First element: " + names.get(0));

// Get the element at index 2.
System.out.println("Element at index 2: " + names.get(2));

// Get the last element using size() - 1.
System.out.println("Last element: " + names.get(names.size() - 1));

// ============================================================
// SECTION 4: SIZE – size()
// ============================================================

System.out.println("\n=== SIZE ===");

// size() returns the number of elements in the list.
System.out.println("Number of names: " + names.size());

// ============================================================
// SECTION 5: CHECKING FOR ELEMENTS – contains()
// ============================================================

System.out.println("\n=== CONTAINS ===");

// Check if "Bob" is in the list.
System.out.println("Contains Bob? " + names.contains("Bob"));
```

```
// Check if "Eve" is in the list.
System.out.println("Contains Eve? " + names.contains("Eve"));

// ============================================================
// SECTION 6: FINDING INDEX – indexOf()
// ============================================================

System.out.println("\n=== INDEX OF ===");

// Find the index of "Charlie".
System.out.println("Index of Charlie: " + names.indexOf("Charlie"));

// Find the index of a non-existent element returns -1.
System.out.println("Index of Eve: " + names.indexOf("Eve"));

// ============================================================
// SECTION 7: REMOVING ELEMENTS – remove()
// ============================================================

System.out.println("\n=== REMOVING ELEMENTS ===");

// Remove by object (name).
names.remove("Bob");

System.out.println("After removing Bob: " + names);
```

```
// Remove by index.
names.remove(0);

System.out.println("After removing element at index 0: " + names);

// ============================================================
// SECTION 8: UPDATING ELEMENTS – set()
// ============================================================

System.out.println("\n=== UPDATING ELEMENTS ===");

// Replace element at index 0 with "Eve".
names.set(0, "Eve");

System.out.println("After setting index 0 to Eve: " + names);

// ============================================================
// SECTION 9: ITERATING WITH A FOR LOOP
// ============================================================

System.out.println("\n=== ITERATING WITH TRADITIONAL FOR LOOP ===");

// Loop through the ArrayList by index.
for (int i = 0; i < names.size(); i++) {
    System.out.println("Index " + i + ": " + names.get(i));
}
```

```
// ============================================================
// SECTION 10: ITERATING WITH ENHANCED FOR LOOP
// ============================================================

System.out.println("\n=== ITERATING WITH ENHANCED FOR LOOP ===");

// For-each loop directly accesses each element.
for (String name : names) {
   System.out.println("Name: " + name);
}

// ============================================================
// SECTION 11: CLEARING THE LIST – clear()
// ============================================================

System.out.println("\n=== CLEARING THE LIST ===");

// Remove all elements from the list.
names.clear();

System.out.println("After clearing: " + names);
System.out.println("Size after clearing: " + names.size());

// ============================================================
// SECTION 12: ARRAYLIST OF INTEGERS (WRAPPER CLASSES)
```

```
        // ============================================================

        System.out.println("\n=== ARRAYLIST OF INTEGERS ===");

        // ArrayList cannot store primitive int; use Integer (wrapper class).
        ArrayList<Integer> scores = new ArrayList<>();

        // Autoboxing automatically converts int to Integer.
        scores.add(85);
        scores.add(92);
        scores.add(78);

        System.out.println("Scores: " + scores);

        // Unboxing automatically converts Integer to int.
        int firstScore = scores.get(0);
        System.out.println("First score: " + firstScore);

        // Sum all scores.
        int total = 0;
        for (int score : scores) {
            total += score;
        }
        System.out.println("Total: " + total);
    }
}
```

Step 4.3: Run the Program

Save and run. Observe the output.

Expected Output:

text

=== ADDING ELEMENTS ===

After adding: [Alice, Bob, Charlie]

After adding Diana at index 1: [Alice, Diana, Bob, Charlie]

=== ACCESSING ELEMENTS ===

First element: Alice

Element at index 2: Bob

Last element: Charlie

=== SIZE ===

Number of names: 4

=== CONTAINS ===

Contains Bob? true

Contains Eve? false

=== INDEX OF ===

Index of Charlie: 3

Index of Eve: -1

=== REMOVING ELEMENTS ===

After removing Bob: [Alice, Diana, Charlie]

After removing element at index 0: [Diana, Charlie]

=== UPDATING ELEMENTS ===

After setting index 0 to Eve: [Eve, Charlie]

=== ITERATING WITH TRADITIONAL FOR LOOP ===

Index 0: Eve

Index 1: Charlie

=== ITERATING WITH ENHANCED FOR LOOP ===

Name: Eve

Name: Charlie

=== CLEARING THE LIST ===

After clearing: []

Size after clearing: 0

=== ARRAYLIST OF INTEGERS ===

Scores: [85, 92, 78]

First score: 85

Total: 255

Step 4.4: Answer These Questions

1. What method adds an element to an ArrayList?
2. What method removes an element by index?
3. What method returns the number of elements?

4. Why can't ArrayList store primitive int directly?
5. What is the difference between add() and set()?

✅ **Task 4 Complete**

- □ I created ArrayListBasics.java
- □ I ran the program successfully
- □ I answered the questions

TASK 5: ArrayList vs Array – Trade-off Analysis

Objective

Evaluate the trade-offs between arrays and ArrayLists by implementing the same task both ways.

Step 5.1: Create the File

Create a new file called ArrayVsArrayList.java in the week5 folder.

Step 5.2: Write the Code

java

```
// This program compares arrays and ArrayLists by implementing
// the same task (managing student names) using both data structures.


// Import Scanner for input and ArrayList for the list.
import java.util.Scanner;
import java.util.ArrayList;


public class ArrayVsArrayList {
    public static void main(String[] args) {


        System.out.println("=== ARRAY vs ARRAYLIST COMPARISON ===\n");
```

```
// ============================================================
// PART 1: ARRAY – FIXED SIZE
// ============================================================

System.out.println("--- ARRAY APPROACH ---");

// Arrays have a fixed size that must be known at creation.
String[] arrayStudents = new String[3];

// Assign values by index.
arrayStudents[0] = "Alice";
arrayStudents[1] = "Bob";
arrayStudents[2] = "Charlie";

// Print the array contents.
System.out.println("Array contents:");
for (String name : arrayStudents) {
   System.out.println("  " + name);
}

// The length is fixed at 3.
System.out.println("Array length: " + arrayStudents.length);

// Cannot add a 4th student without creating a new array.
System.out.println("Cannot add a 4th element without creating a new array.\n");
```

```
// ============================================================
// PART 2: ARRAYLIST – DYNAMIC SIZE
// ============================================================

System.out.println("--- ARRAYLIST APPROACH ---");

// ArrayLists don't require a size; they grow dynamically.
ArrayList<String> listStudents = new ArrayList<>();

// Add values using add().
listStudents.add("Alice");
listStudents.add("Bob");
listStudents.add("Charlie");

// Print the ArrayList contents.
System.out.println("ArrayList contents:");
for (String name : listStudents) {
   System.out.println("  " + name);
}

// Size can change.
System.out.println("ArrayList size: " + listStudents.size());

// Add a 4th student easily.
listStudents.add("Diana");
```

```
System.out.println("After adding Diana, size: " + listStudents.size());

// Remove a student easily.
listStudents.remove("Bob");
System.out.println("After removing Bob, size: " + listStudents.size());
System.out.println("Updated contents: " + listStudents + "\n");

// ============================================================
// PART 3: TRADE-OFF SUMMARY TABLE
// ============================================================

System.out.println("=== TRADE-OFF COMPARISON ===");
System.out.println("Feature          | Array       | ArrayList");
System.out.println("-----------------|-------------|-----------");
System.out.println("Size             | Fixed       | Dynamic");
System.out.println("Data types       | Primitives & Objects | Objects only");
System.out.println("Syntax for length | .length     | .size()");
System.out.println("Add element      | Assign by index | .add()");
System.out.println("Remove element   | Manual shift | .remove()");
System.out.println("Memory efficiency | Higher      | Lower (overhead)");
System.out.println("Performance      | Faster access| Slower access");
System.out.println("Best for         | Fixed-size data | Changing-size data");

// ============================================================
// PART 4: PRACTICAL DECISION GUIDE
// ============================================================
```

```
        System.out.println("\n=== WHEN TO USE WHICH ===");
        System.out.println("Use ARRAY when:");
        System.out.println("  - The number of elements is known and fixed");
        System.out.println("  - You need maximum performance");
        System.out.println("  - You are storing primitives");

        System.out.println("\nUse ARRAYLIST when:");
        System.out.println("  - The number of elements may change");
        System.out.println("  - You need methods like add, remove, contains");
        System.out.println("  - You are storing objects");
    }
}
```

Step 5.3: Run the Program

Save and run. Observe the output.

Step 5.4: Answer These Questions

1. What is the main limitation of arrays?
2. What is the main advantage of ArrayLists?
3. Which has better performance? Why?
4. In what scenario would you prefer an array over an ArrayList?
5. In what scenario would you prefer an ArrayList over an array?

✅ **Task 5 Complete**

- □ I created ArrayVsArrayList.java
- □ I ran the program successfully
- □ I answered the questions

TASK 6: Debugging Array Index Errors

Objective

Debug ArrayIndexOutOfBoundsException by tracing loop conditions.

Step 6.1: Examine the Buggy Code

Create a file called ArrayIndexDebug.java in the week5 folder.

java

```
// This program contains array index errors. Can you find and fix them?


public class ArrayIndexDebug {
    public static void main(String[] args) {


        // Declare an array of 5 elements.
        int[] numbers = {10, 20, 30, 40, 50};


        // ============================================================
        // BUG 1: Off-by-one error in loop condition
        // ============================================================


        System.out.println("=== BUG 1 ===");


        // The intention: print all 5 elements.
        // BUG: i <= numbers.length allows i to reach 5.
        // Valid indices are 0, 1, 2, 3, 4. Index 5 is out of bounds.
        // FIX: Change <= to <.
        for (int i = 0; i <= numbers.length; i++) {
            System.out.println("numbers[" + i + "] = " + numbers[i]);
```

```
    }

    // ============================================================
    // BUG 2: Accessing invalid index directly
    // ============================================================

    System.out.println("\n=== BUG 2 ===");

    // The intention: access the last element.
    // BUG: numbers[5] is out of bounds for a length-5 array.
    // FIX: Use numbers.length - 1 instead of a hardcoded value.
    System.out.println("Last element: " + numbers[5]);

    // ============================================================
    // BUG 3: Negative index
    // ============================================================

    System.out.println("\n=== BUG 3 ===");

    // The intention: access the first element.
    // BUG: Index -1 is invalid; arrays start at index 0.
    // FIX: Use index 0.
    System.out.println("First element: " + numbers[-1]);
  }
}
```

Step 6.2: Identify the Bugs

Bug 1:

- **Problem:** i <= numbers.length allows i to reach numbers.length (which is 5).
- **Fix:** Change <= to < (so i goes from 0 to 4).

Bug 2:

- **Problem:** Hardcoded index 5 is out of bounds for an array of length 5.
- **Fix:** Use numbers.length - 1 (which is 4).

Bug 3:

- **Problem:** Index -1 is invalid; arrays start at index 0.
- **Fix:** Use index 0.

Step 6.3: Create the Fixed Version

Create a new file called FixedArrays.java:

java

```
// This program contains the corrected versions of the array bugs.


public class FixedArrays {
    public static void main(String[] args) {

        // Declare an array of 5 elements.
        int[] numbers = {10, 20, 30, 40, 50};

        // ============================================================
        // FIXED BUG 1: Print all elements
        // ============================================================

        System.out.println("=== FIXED BUG 1 ===");
```

```
    // FIX: Change <= to <. Now i goes from 0 to 4.
    for (int i = 0; i < numbers.length; i++) {
        System.out.println("numbers[" + i + "] = " + numbers[i]);
    }

    // ============================================================
    // FIXED BUG 2: Access last element
    // ============================================================

    System.out.println("\n=== FIXED BUG 2 ===");

    // FIX: Use numbers.length - 1 to get the last valid index.
    System.out.println("Last element: " + numbers[numbers.length - 1]);

    // ============================================================
    // FIXED BUG 3: Access first element
    // ============================================================

    System.out.println("\n=== FIXED BUG 3 ===");

    // FIX: Use index 0 to get the first element.
    System.out.println("First element: " + numbers[0]);

    System.out.println("\nAll arrays accessed safely!");
}
```

}

Step 6.4: Run the Fixed Program

Save and run. Verify all bugs are fixed.

Expected Output:

text

=== FIXED BUG 1 ===

numbers[0] = 10

numbers[1] = 20

numbers[2] = 30

numbers[3] = 40

numbers[4] = 50

=== FIXED BUG 2 ===

Last element: 50

=== FIXED BUG 3 ===

First element: 10

All arrays accessed safely!

Step 6.5: Answer These Questions

1. What is the valid index range for an array of length n?
2. What exception is thrown when you access an invalid index?
3. How do you safely access the last element of an array?
4. What is the correct loop condition to iterate through all elements?
5. How can you prevent array index errors before they occur?

✅ **Task 6 Complete**

- □ I identified all three bugs
- □ I created FixedArrays.java
- □ I ran the fixed program successfully
- □ I answered the questions

TASK 7: Integrated Project – Student Roster

Objective

Design an application that uses both arrays and ArrayLists to manage a student roster.

Step 7.1: Create the File

Create a new file called StudentRoster.java in the week5 folder.

Step 7.2: Write the Code

java

```
// This program integrates arrays and ArrayLists to manage a student roster.
// It uses an array for fixed data (subjects) and an ArrayList for dynamic
// data (student names and marks).


// Import Scanner and ArrayList.
import java.util.Scanner;
import java.util.ArrayList;


public class StudentRoster {
    public static void main(String[] args) {


        // Create a Scanner object for user input.
        Scanner scanner = new Scanner(System.in);
```

```
// ============================================================
// SECTION 1: FIXED DATA – SUBJECTS (ARRAY)
// ============================================================

// The subjects offered are fixed; an array is appropriate.
String[] subjects = {"Math", "English", "Science"};

// ============================================================
// SECTION 2: DYNAMIC DATA – STUDENTS (ARRAYLIST)
// ============================================================

// Student names and marks may grow, so use ArrayLists.
ArrayList<String> studentNames = new ArrayList<>();
ArrayList<int[]> studentMarks = new ArrayList<>();

// ============================================================
// SECTION 3: ADD INITIAL STUDENTS
// ============================================================

// Add sample students (name and marks array).
studentNames.add("Alice");
studentMarks.add(new int[]{85, 90, 78});

studentNames.add("Bob");
studentMarks.add(new int[]{72, 88, 91});
```

```
studentNames.add("Charlie");
studentMarks.add(new int[]{65, 75, 80});

// ============================================================
// SECTION 4: MAIN MENU (do-while loop)
// ============================================================

int choice;

do {
    System.out.println("\n==========================================");
    System.out.println("       STUDENT ROSTER MANAGER");
    System.out.println("==========================================");
    System.out.println("1. Add New Student");
    System.out.println("2. View All Students");
    System.out.println("3. View Class Average");
    System.out.println("4. Search Student by Name");
    System.out.println("5. Remove Student");
    System.out.println("6. Exit");
    System.out.println("==========================================");
    System.out.print("Enter your choice (1-6): ");

    // Read the choice.
    choice = scanner.nextInt();
    scanner.nextLine(); // Consume newline
```

```
// ============================================================
// SWITCH: Handle the user's choice.
// ============================================================

switch (choice) {

    case 1:
        // Add a new student.
        System.out.print("Enter student name: ");
        String name = scanner.nextLine();

        // Create an array for this student's marks.
        int[] marks = new int[subjects.length];

        // Loop through subjects and read marks.
        for (int i = 0; i < subjects.length; i++) {
            System.out.print("Enter mark for " + subjects[i] + ": ");
            marks[i] = scanner.nextInt();
        }
        scanner.nextLine(); // Consume newline

        // Add student to the lists.
        studentNames.add(name);
        studentMarks.add(marks);
```

```
    System.out.println("✓ Student '" + name + "' added successfully.");
    break;

case 2:
    // View all students.
    System.out.println("\n=== STUDENT ROSTER ===");
    if (studentNames.isEmpty()) {
        System.out.println("No students in the roster.");
    } else {
        // Print table header.
        System.out.printf("%-12s", "Student");
        for (String subject : subjects) {
            System.out.printf("%-12s", subject);
        }
        System.out.println("Average");
        System.out.println("-----------------------------------------------");

        // Loop through all students.
        for (int i = 0; i < studentNames.size(); i++) {
            System.out.printf("%-12s", studentNames.get(i));

            int total = 0;
            int[] marks = studentMarks.get(i);

            // Loop through marks for this student.
            for (int mark : marks) {
```

```
                System.out.printf("%-12d", mark);
                total += mark;
            }

            // Calculate and print the average.
            double avg = (double) total / marks.length;
            System.out.printf("%.2f%n", avg);
        }
    }
    break;

case 3:
    // View class average.
    if (studentNames.isEmpty()) {
        System.out.println("No students to calculate average.");
    } else {
        int grandTotal = 0;
        int totalMarks = 0;

        // Loop through all students and sum their marks.
        for (int[] marks : studentMarks) {
            for (int mark : marks) {
                grandTotal += mark;
                totalMarks++;
            }
        }
```

```
            double classAvg = (double) grandTotal / totalMarks;
            System.out.printf("Class Average: %.2f%n", classAvg);
        }
        break;

    case 4:
        // Search for a student.
        System.out.print("Enter name to search: ");
        String searchName = scanner.nextLine();

        // Find the index of the student.
        int index = studentNames.indexOf(searchName);

        if (index != -1) {
            // Student found; print details.
            System.out.println("\n=== STUDENT FOUND ===");
            System.out.println("Name: " + studentNames.get(index));
            int[] marks = studentMarks.get(index);
            for (int i = 0; i < subjects.length; i++) {
                System.out.println(subjects[i] + ": " + marks[i]);
            }
        } else {
            System.out.println("✗ Student '" + searchName + "' not found.");
        }
        break;
```

```
case 5:
   // Remove a student.
   System.out.print("Enter name to remove: ");
   String removeName = scanner.nextLine();

   int removeIndex = studentNames.indexOf(removeName);

   if (removeIndex != -1) {
      studentNames.remove(removeIndex);
      studentMarks.remove(removeIndex);
      System.out.println("✓ Student '" + removeName + "' removed.");
   } else {
      System.out.println("✗ Student '" + removeName + "' not found.");
   }
   break;

case 6:
   // Exit the program.
   System.out.println("Thank you for using the Student Roster Manager!");
   break;

default:
   // Invalid choice.
   System.out.println("✗ Invalid choice. Please enter 1-6.");
   break;
```

```
            }


        } while (choice != 6);


        // Close the scanner.
        scanner.close();
    }
}
```

Step 7.3: Run the Program

Save and run. Test all menu options:

1. View all students (option 2)
2. Add a new student (option 1)
3. View class average (option 3)
4. Search for a student (option 4)
5. Remove a student (option 5)
6. Exit (option 6)

Sample Interaction:

```
==============================================
      STUDENT ROSTER MANAGER
==============================================
1. Add New Student
2. View All Students
3. View Class Average
4. Search Student by Name
5. Remove Student
6. Exit
```

==========

Enter your choice (1-6): 2

=== STUDENT ROSTER ===

Student Math English Science Average

Alice 85 90 78 84.33

Bob 72 88 91 83.67

Charlie 65 75 80 73.33

Step 7.4: Answer These Questions

1. Why are subjects stored in an array instead of an ArrayList?
2. Why are student names stored in an ArrayList?
3. How does the program handle adding a new student?
4. What happens when you search for a student who doesn't exist?
5. How would you modify the program to store multiple classes?

✅ **Task 7 Complete**

- □ I created StudentRoster.java
- □ I tested all menu options
- □ I answered the questions

TASK 8: Push to GitHub

Step 8.1: Stage and Commit

1. Open the **Source Control** panel in VS Code.
2. Stage all new files in week5.
3. Commit with message: Week 5: Arrays and ArrayLists - basics, algorithms, 2D arrays, comparison

4. Push to GitHub.

Step 8.2: Verify on GitHub

Go to your repository and confirm all Week 5 files are there.

✅ **Task 8 Complete**

- □ I committed and pushed all files
- □ I verified on GitHub

➢ **Final Self-Assessment Checklist**

- □ I can declare, initialize, and access arrays
- □ I can iterate through arrays using for and enhanced for loops
- □ I can implement array algorithms (max, min, average, search)
- □ I can work with 2D arrays
- □ I can create and use ArrayLists
- □ I can add, remove, access, and update ArrayList elements
- □ I can compare arrays and ArrayLists and choose the right one
- □ I can debug array index out-of-bounds errors
- □ I pushed all Week 5 code to GitHub

➢ **Troubleshooting Guide**

Problem	Solution
ArrayIndexOutOfBoundsException	Check loop condition uses < not <=; check index bounds
NullPointerException	Ensure array/ArrayList is initialized before use
cannot find symbol: ArrayList	Add import java.util.ArrayList;

Problem	Solution
Cannot add primitive to ArrayList	Use wrapper class (Integer, Double)
ConcurrentModificationException	Don't modify ArrayList while iterating with for-each
InputMismatchException	You entered text instead of a number

Reflection Questions

1. Which data structure did you find more useful: arrays or ArrayLists? Why?
2. What was the most challenging part of this practical session?
3. What error message did you encounter, and how did you solve it?
4. In your own words, when would you use an array vs an ArrayList?
5. How will arrays and ArrayLists help you build real-world applications?

What You've Achieved

You have just:

- Mastered **array declaration, initialization, and access**
- Implemented **array algorithms** (max, min, average, search)
- Used **2D arrays** for tabular data
- Created and manipulated **ArrayLists**
- **Compared** arrays and ArrayLists
- **Debugged** index out-of-bounds errors
- Built an **integrated application** (Student Roster)

You can now store and manage collections of data!

➢ **Next Steps**

In Week 6, you will:

1. Dive deeper into **classes and objects**
2. Learn about **constructors** and **encapsulation**
3. Build a **BankAccount** class and a **Book** class
4. Continue pushing your code to GitHub

Ø **Additional Resources**

Topic	Resource
Arrays	https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html
ArrayList	https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html
Big-O Analysis	https://www.geeksforgeeks.org/analysis-algorithms-big-o-analysis/

END OF PRACTICAL SESSION 5

"Programs must be written for people to read, and only incidentally for machines to execute." – Harold Abelson

---

Instructor Note: You should complete this lab independently. I encourage experimentation with different array sizes and algorithms. The goal is to build intuition for when to use arrays vs ArrayLists.